Detail 1:

```
y->hauteur = max(obtenirHauteur(y->gauche), obtenirHauteur(y->droite)) + 1;
```

Cette ligne calcule et met à jour la **hauteur** du noeud actuel dans l'arbre AVL.

Explication simple:

La hauteur d'un noeud = 1 + la hauteur maximale de ses deux enfants (gauche et droite)

Décomposition de la ligne:

noeud->hauteur = 1 + max(obtenirHauteur(noeud->gauche), obtenirHauteur(noeud->droite));

obtenirHauteur(noeud->gauche) → Obtient la hauteur du fils gauche

obtenirHauteur(noeud->droite) → Obtient la hauteur du fils droit

max(...) → Prend la plus grande des deux hauteurs

1 + → Ajoute 1 pour compter le noeud actuel

noeud->hauteur = → Stocke le résultat dans le noeud

Exemple visuel:

```
    A (hauteur = 3)
    / \
   B   C (hauteur = 1)
  / \
 D   E (hauteur = 1)
```

Noeud D: hauteur = 1 + max(0, 0) = 1 (pas d'enfants)

Noeud E: hauteur = 1 + max(0, 0) = 1 (pas d'enfants)

Noeud B: hauteur = 1 + max(1, 1) = 2 (enfants D et E ont hauteur 1)

Noeud C: hauteur = 1 + max(0, 0) = 1 (pas d'enfants)

Noeud A: hauteur = 1 + max(2, 1) = 3 (enfant gauche B a hauteur 2)

Pourquoi c'est important dans un AVL?

L'arbre AVL utilise la hauteur pour calculer le **facteur d'équilibre** et décider si des rotations sont nécessaires pour garder l'arbre équilibré.

Detail2:

```
int obtenirBalance(NoeudAVL* n) {
    if (n == NULL) {
        return 0;
    }
    else{
        return obtenirHauteur(n->gauche) - obtenirHauteur(n-
>droite);
    }
}
```

Le facteur d'équilibre = hauteur du sous-arbre gauche - hauteur du sous-arbre droit

Interprétation des valeurs

- **Balance = 0** : l'arbre est parfaitement équilibré
- **Balance = 1** : le sous-arbre gauche est légèrement plus haut (acceptable)
- **Balance = -1** : le sous-arbre droit est légèrement plus haut (acceptable)
- **Balance ≥ 2** : déséquilibre à gauche → nécessite une rotation
- **Balance ≤ -2** : déséquilibre à droite → nécessite une rotation

Exemple
```
    10 (balance = 1)
   / \
  5   15
 /
3
```

Detail 3:

Rotation Droite

```
NoeudAVL* rotationDroite(NoeudAVL* y) {
    NoeudAVL* x = y->gauche; // x devient la nouvelle racine
    NoeudAVL* T2 = x->droite;// Sauvegarde du sous-arbre T2

    // Effectuer la rotation

    x->droite = y;     // y devient enfant droit de x
    y->gauche = T2;    // T2 devient enfant gauche de y

    // Mettre à jour les hauteurs (y d'abord car x dépend de y)

    y->hauteur = max(obtenirHauteur(y->gauche),
obtenirHauteur(y->droite)) + 1;
    x->hauteur = max(obtenirHauteur(x->gauche),
obtenirHauteur(x->droite)) + 1;

    return x;  // x est la nouvelle racine

}
```

Avant rotation :
```
      y
     / \
    x   T3
   / \
  T1  T2
```

Après rotation :
```
      x
     / \
    T1  y
       / \
      T2  T3
```

Rotation Gauche

```
NoeudAVL* rotationGauche(NoeudAVL* x) {
   NoeudAVL* y = x->droite;  // y devient la nouvelle racine
   NoeudAVL* T2 = y->gauche; // Sauvegarde du sous-arbre T2

   // Effectuer la rotation
   y->gauche = x;   // x devient enfant gauche de y
   x->droite = T2;  // T2 devient enfant droit de x

   // Mettre à jour les hauteurs
   x->hauteur = max(obtenirHauteur(x->gauche),
obtenirHauteur(x->droite)) + 1;
   y->hauteur = max(obtenirHauteur(y->gauche),
obtenirHauteur(y->droite)) + 1;

   return y;  // y est la nouvelle racine
}
```

Avant rotation :
```
    x
   / \
  T1  y
     / \
    T2  T3
```

Après rotation :
```
      y
     / \
    x   T3
   / \
  T1  T2
```

Cas d'utilisation dans un AVL

Ces rotations sont utilisées dans 4 cas de déséquilibre :

1. **Cas Gauche-Gauche (LL)** : Rotation droite simple
2. **Cas Droite-Droite (RR)** : Rotation gauche simple
3. **Cas Gauche-Droite (LR)** : Rotation gauche sur enfant gauche, puis rotation droite
4. **Cas Droite-Gauche (RL)** : Rotation droite sur enfant droit, puis rotation gauche

Detail 4:

Une fonction qui ajoute une **nouvelle publication** à la liste des publications d'un utilisateur

```
void ajouterPublication(Utilisateur* u, const char* texte) {
   if (!u) return;  // Vérification de sécurité

   // 1. Allouer mémoire pour la nouvelle publication
   Publication* pub = malloc(sizeof(Publication));

   // 2. Remplir les champs de la publication
   pub->auteur_id = u->id;
   strncpy(pub->texte, texte, MAX_MSG - 1);
   pub->texte[MAX_MSG - 1] = '\0';  // Assurer la terminaison
   pub->timestamp = time(NULL);      // Horodatage actuel

   // 3. Insertion en tête de liste chaînée
   pub->suivant = u->publications;
   u->publications = pub;

   printf("--- Publication ajoutee!\n");
}
```

1/ Vérification de sécurité :Évite un segmentation faut si le pointeur utilisateur est NULL.

2/ strncpy copie au maximum MAX_MSG - 1 caractères
Force la terminaison par «\0» pour éviter les débordements

3 / pub->timestamp = time(NULL);
Enregistre l'heure actuelle en secondes (1er janvier 1970).

4/ pub->suivant = u->publications; // Lien vers l'ancienne tête
u->publications = pub; // Nouvelle tête = pub

Les publications les plus récentes apparaissent en premier (LIFO).

Detail 4:

Cette fonction crée une **copie profonde récursive** d'une liste chaînée de publications

```
Publication* copierPile(Publication* p) {
   if (!p) return NULL;  // Cas de base : fin de liste

   // 1. Allouer mémoire pour le nouveau nœud
   Publication* copie = malloc(sizeof(Publication));

   // 2. Copier tous les champs de la publication originale
   *copie = *p;

   // 3. Appel récursif pour copier le reste de la liste
   copie->suivant = copierPile(p->suivant);

   return copie;  // Retourner la tête de la copie
}
```

Exemple visuel

Liste originale :

p -> [A] -> [B] -> [C] -> NULL

Exécution récursive :

```
copierPile([A])
  ├─ malloc nouveau [A']
  ├─ copie [A'] = [A]
  └─ [A']->suivant = copierPile([B])
       ├─ malloc nouveau [B']
       ├─ copie [B'] = [B]
       └─ [B']->suivant = copierPile([C])
            ├─ malloc nouveau [C']
            ├─ copie [C'] = [C]
            └─ [C']->suivant = copierPile(NULL)
                 └─ return NULL
```

Résultat :

p -> [A] -> [B] -> [C] -> NULL (original)

copie -> [A'] -> [B'] -> [C'] -> NULL (copie indépendante)

Detail5:

Cette fonction supprime un utilisateur d'un **arbre AVL** tout en maintenant l'équilibre de l'arbre

Structure globale

```
NoeudAVL* supprimerUtilisateur(NoeudAVL* racine, int id) {
   // 1. Recherche récursive du nœud
   // 2. Suppression du nœud (3 cas possibles)
   // 3. Mise à jour de la hauteur
   // 4. Rééquilibrage AVL (4 cas de rotation)
}
```

Partie 1 : Recherche récursive

```
if (!racine) return NULL;  // Arbre vide ou nœud non trouvé

if (id < racine->utilisateur.id)
    racine->gauche = supprimerUtilisateur(racine->gauche, id);
else if (id > racine->utilisateur.id)
    racine->droite = supprimerUtilisateur(racine->droite, id);
else {
    // Nœud trouvé ! (section suivante)
}
```

Partie 2 : Suppression (3 cas)

Cas 1 & 2 : Nœud avec 0 ou 1 enfant

```
if (!racine->gauche || !racine->droite) {
    // Récupérer l'enfant existant (ou NULL si aucun)
    NoeudAVL* temp = racine->gauche ? racine->gauche : racine->droite;

    // Libérer toutes les ressources de l'utilisateur
    libererListe(racine->utilisateur.amis);
    libererListe(racine->utilisateur.abonnements);
    libererPublications(racine->utilisateur.publications);
    free(racine);

    return temp;  // Remplacer le nœud par son enfant
}
```

Exemple :

```
Avant:        10              Après:       15
             /  \                         /  \
           5    15 (supprimer)          12   20
               /  \
             12   20
```

Cas 3 : Nœud avec 2 enfants

```
// Trouver le successeur (minimum du sous-arbre droit)
NoeudAVL* temp = trouverMinimum(racine->droite);

// Copier les données du successeur
racine->utilisateur = temp->utilisateur;

// Supprimer le successeur (qui a au plus 1 enfant)
racine->droite = supprimerUtilisateur(racine->droite, temp-
>utilisateur.id);
```

Exemple :

```
Avant:          20                  Après:          25
               /  \                               /  \
             10    30 (supprimer)               10    30
                 /  \                                /  \
                25    40                           28    40
                  \
                  28

Successeur de 30 = 25 (minimum à droite)
```

Partie 3 : Mise à jour de la hauteur

```
if (!racine) return NULL;  // Sécurité après suppression
racine->hauteur = 1 + max(obtenirHauteur(racine->gauche),
                          obtenirHauteur(racine->droite));
int balance = obtenirBalance(racine);
```

Partie 4 : Rééquilibrage (4 cas)

Cas 1 : Gauche-Gauche (LL)

```
if (balance > 1 && obtenirBalance(racine->gauche) >= 0)
    return rotationDroite(racine);
```

```
      z (balance=2)                y
     / \                          / \
    y   T4    ————————>           x   z
   / \          Rotation        / \ / \
  x   T3        Droite        T1 T2 T3 T4
 / \
T1  T2
```

Cas 2 : Gauche-Droite (LR)

```
if (balance > 1 && obtenirBalance(racine->gauche) < 0) {
    racine->gauche = rotationGauche(racine->gauche);  // Rotation
gauche sur y
    return rotationDroite(racine);                    // Rotation
droite sur z
```

```
}
```

```
     z                     z                     x
    / \                   / \                   / \
   y   T4    ---------->   x   T4    ---------->   y   z
  / \        Rot.Gauche / \        Rot.Droite/ \ / \
 T1  x       sur y     y  T3       sur z    T1 T2 T3 T4
    / \               / \
   T2  T3           T1  T2
```

Cas 3 : Droite-Droite (RR)

```
if (balance < -1 && obtenirBalance(racine->droite) <= 0)
    return rotationGauche(racine);
```

```
  z (balance=-2)              y
 / \                         / \
T1  y        ---------->     z   x
   / \       Rotation      / \ / \
  T2  x      Gauche      T1 T2 T3 T4
     / \
    T3  T4
```

Cas 4 : Droite-Gauche (RL)

```
if (balance < -1 && obtenirBalance(racine->droite) > 0) {
    racine->droite = rotationDroite(racine->droite);  // Rotation
droite sur y
    return rotationGauche(racine);                    // Rotation
gauche sur z
}
```

```
  z                     z                     x
 / \                   / \                   / \
T1  y     ---------->  T1  x     ---------->   z   y
   / \    Rot.Droite  / \     Rot.Gauche/ \ / \
  x  T4   sur y      T2  y     sur z    T1 T2 T3 T4
 / \                    / \
T2  T3                 T3  T4
```